

Positioning File Pointer

To random access files, C requires a function with the help of which the file pointer can be positioned at any random location in a file. Such a function defined in the standard library is:



The fseek() Function

```
int fseek(FILE *fp, long offset, int pos);
```

The first argument is a pointer to a file. The second argument is the number of bytes to move the file pointer, counting from zero. This argument can be positive, negative or zero depending on



the desired movement. The third parameter is a flag indicating from where in the file to compute the offset.

It can have three values:

- `SEEK_SET` (or value 0) - beginning of file
- `SEEK_CUR` (or value 1) - current position
- `SEEK_END` (or value 2) - end of file

These three constants are defined in `<stdio.h>`. If successful `fseek()` returns zero. Another function `rewind()` is used to reset the file position to the beginning of the file. Its prototype is:

```
void rewind(FILE *fp);
```

Another function `ftell()` is used to tell the position of the file pointer. Its prototype is:

```
long ftell(FILE *fp);
```

It returns -1 on error and the position of the file pointer if successful.

1/4

Opening and Closing Files

The function used to open a file is `open()`. Its prototype is:

```
int open(char *filename, int mode, int access);
```

Here mode indicates one of the following macros in .

- `O_RDONLY` - Read only
- `O_WRONLY` - Write only
- `O_RDWR` - Read / Write

The access parameter is used in UNIX environment for providing access to particular users and is just included here for compatibility and can be set to zero. `open()` function returns `-1` on failure.

If the file does not exist, `open()` function will not create it. The function `creat()` is used which will create new files and re-write old ones. The prototype is:

```
int creat(char *filename, int access);
```

It returns a file descriptor; if successful else it returns `-1`. It is not an error to create an already existing file, the function will just truncate its length to zero.

The function `close()` is used to close a file. The prototype is:

```
int close(int fd);
```

It returns zero if the file is closed successfully or `-1` if not. If function `fclose()` is not called explicitly,

the operating system normally will close the file when the program execution terminates.

2/4

Reading, Writing and Positioning

The functions `read()` and `write()` are used to read from and write to a file. The prototypes are:

```
int read(int fd, void *buf, int size);  
int write(int fd, void *buf, int size);
```

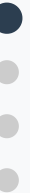
The first parameter is the file descriptor returned by `open()`, the second parameter holds the data which must be typecast to the format needed by the program, and the third parameter indicates the number of bytes to be transferred. The return value tells how many bytes are actually transferred. If this value is `-1`, then an error must have occurred.

File System Types

The file pointers assigned to standard input, standard output and standard error are manipulated using file pointers `stdin`, `stdout`, and `stderr`.

The standard input, standard output and standard error are manipulated using file pointers `stdin`, `stdout`, and `stderr`.

The buffered I/O system uses buffered input and output, that is, operating system handles details of data retrieval and storage, system stores data temporarily (buffers it) in order to optimize file system access.



The buffered I/O functions are handled directly as system calls without buffering by the operating system. That is why they are also known as low level functions. This is referred to as unbuffered I/O system because the programmer must provide and maintain all disk buffers, routines do not do it automatically.

The low level functions are defined in header file .